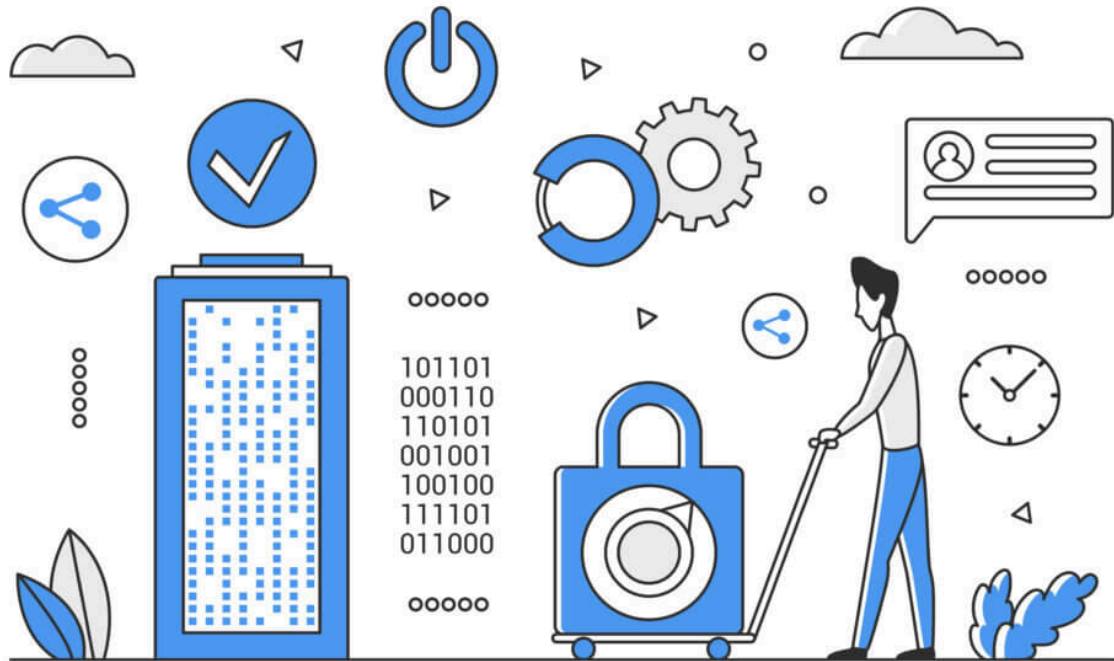


## 2.2 performance testing:

- Understanding load testing, stress testing and scalability testing?



## What is Performance Testing?

On the other hand, performance testing is a type of non-functional software testing used to verify the performance of a system under some condition. It focuses on some major factors for measuring the performance level of the software application under the expected workload; the factors are speed, scalability, reliability, and resource usage. The main focus is the identification and removal of performance bottlenecks, which may cause bad user experience or system stability. Very important for applications whose success is in the response time and stability under load, like e-commerce websites, online banking, and social media sites. Some of the performance testing types that are very common include load testing, stress testing, scalability testing, endurance testing, and others that target different types of the system performance.

## Example of Performance Testing

A small online bakery decides to launch a new line of cakes. They also perform [performance testing](#) on their website, which is expecting huge traffic from enthusiastic customers. It's executed by test cases in which great quantities of users browse through the cake gallery, add cakes to cart, and pass checkouts simultaneously. The initial test will demonstrate slow loading and failures of checkout times when the number of users on-site is more than 50 at once. This will be optimised by compressing the images and upgrading the hosting plan. After this, there should be some more testing—perhaps, next time, looking to verify that the website can handle up to 100 customers at a time. This should ensure a really smooth launch day, where 100 happy customers can ideally place their cake orders within a session.

## What is Load Testing?

Load testing is a technique to test the behaviour and performance of a system under some expected specific load. These basically are simulations that replicate real-life cases whereby the software, application, or site has a very high load of users or transactions, all concurrently accessing it at the same time. This is done purely for the purposes of checking its capacity and scalability. A primary aim in load testing is to find performance bottlenecks. It ensures that the system can handle the increasing amount of user traffic without deterioration of response time, reliability, or user experience.

It does provide an excellent opportunity for developers and engineers to have clear insights regarding the system limits, maximum operating capacity, and any potential problem that might affect users during peak timings.

## Example of Load Testing

A local fitness centre is launching an online membership sign-up feature to handle New Year's resolution traffic. They're predicting an onslaught of people trying to sign up in January, which means they'll have to do load testing. They create the number of virtual users to be anticipated and then increase gradually as they access the sign-up page in order to register and pay for the gym membership. Preliminary tests, having been done based on the basic

assumptions, show that the site started slowing down considerably and even crashing during the processing of more than 100 registrations at a go. After optimising server capacity and streamlining the registration process, additional tests confirmed that the site is now able to handle the expected New Year increase in registrations. What is Stress Testing? Stress testing is an exploration of performance in systems and what kind of performance a system will produce under conditions far above those required for normal operation. The main goal was to find the point of break in the system, and then observe how it copes with an overloaded situation—i.e., how it fails and recovers from the failure gracefully.

## **Stress Testing**

Stress testing submits the system to loads of exaggerated values, very high concurrency, or reduced resources on purpose to bring out vulnerability, stability problems, and probable data loss that may be undetected under a normal environment. It is very important to understand the upper bounds, both at the application point of view and enforcement, along with the data integrity that includes strong error recovery and handling mechanisms.

## **Example of Stress Testing**

A mobile banking app prepares for an upcoming feature release that allows users to buy digital gold. The stress tests ensure that the app has been built to be able to withstand the pressure, considering that with high expectations of demand, pressure is felt. This is done by simulating transactions to a level far higher than that usually experienced, increasing magnitudes with time until the app gives in, through slow responses and failed transactions. The testing pointed out a critical juncture where the system would not be able to process the requests optimally. With such insights, the team makes the app backend infrastructure more robust and refines error-catching mechanisms. This makes the app functional and easy for the user, even when it is running under extreme unexpected conditions.

### Conclusion:

Performance testing, load testing, and stress testing are a few core elements within a well-rounded testing strategy, each touching different dimensions of application resilience and user experience. Performance Testing means the detailed checking of speed, stability, and scalability in the most diverse range of conditions. Load Testing focuses on the behaviour of the application under expected volumes of users, meaning making sure that the application can take its peak usage without degrading performance.

Stress Testing: A process that pushes the system beyond its normal operating thresholds to identify beyond a reasonable doubt the breaking point and observe its recovery mechanisms. All this assures, on the one hand, that applications are optimised for everyday use and, on the other hand, that they are stable and reliable even in the most demanding circumstances, providing a flawless experience to users.